

```
import java.util.LinkedList;
import java.util.Scanner;

class Node {
    int data;
    LinkedList<Node> neighbors;

    Node(int data) {
        this.data = data;
        this.neighbors = new LinkedList<>();
    }

    void addNeighbor(Node neighbor) {
        neighbors.add(neighbor);
    }
}

public class DFS {

    static void dfs(Node root) {
        boolean[] visited = new boolean[100]; // Assume max 100 nodes
        dfsHelper(root, visited);
    }

    static void dfsHelper(Node node, boolean[] visited) {
        if (visited[node.data]) {
            return; // Already visited
        }
        visited[node.data] = true;
        System.out.print(node.data + " "); // Print current node

        for (Node neighbor : node.neighbors) {
            if (!visited[neighbor.data]) {
                dfsHelper(neighbor, visited); // Explore unvisited neighbors
            }
        }
    }
}
```

```
}  
}
```

```
public static void main(String[] args) {  
    Scanner sc = new Scanner(System.in);  
  
    // Read number of nodes and edges  
    System.out.print("Enter number of nodes: ");  
    int n = sc.nextInt();  
    System.out.print("Enter number of edges: ");  
    int m = sc.nextInt();  
  
    // Create nodes and edges  
    Node[] nodes = new Node[n];  
    for (int i = 0; i < n; i++) {  
        nodes[i] = new Node(i);  
    }  
    for (int i = 0; i < m; i++) {  
        int from = sc.nextInt();  
        int to = sc.nextInt();  
        nodes[from].addNeighbor(nodes[to]);  
    }  
  
    // Choose starting node for DFS  
    System.out.print("Enter starting node for DFS: ");  
    int start = sc.nextInt();  
  
    // Perform DFS  
    dfs(nodes[start]);  
  
    sc.close();  
}
```